



# Beyond the Hype: Underappreciated Database Topics that Still Matter

Xuan Zhou

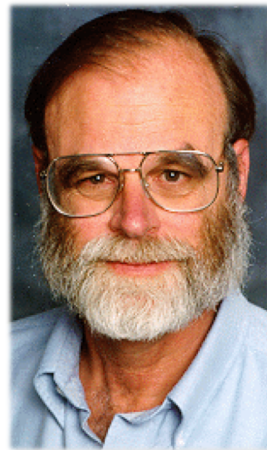
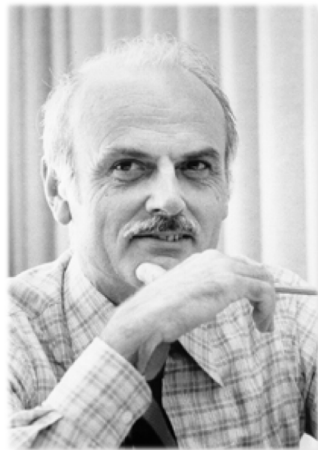
DaSE, East China Normal University

Feb 2026, Japan

# What Causes DB Systems to Evolve ?

- New Applications
  - Waves of WWW and AI → Explosion of Data Volume and Diversity
- New Hardware
  - Cloud Platforms, Emerging Storage Devices, Emerging Processors
- New Practice of Software Engineering
  - Agile / Cloud Native - Software Development, Microservice, EDA

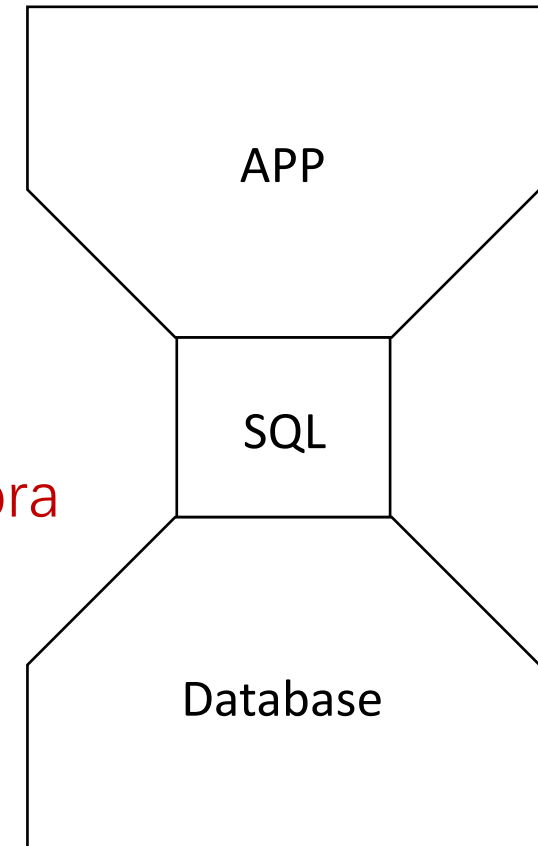
# The History of DBMS in the Eyes of DBers



# The Design Goal of Relational Model: Simplification of Software Engineering

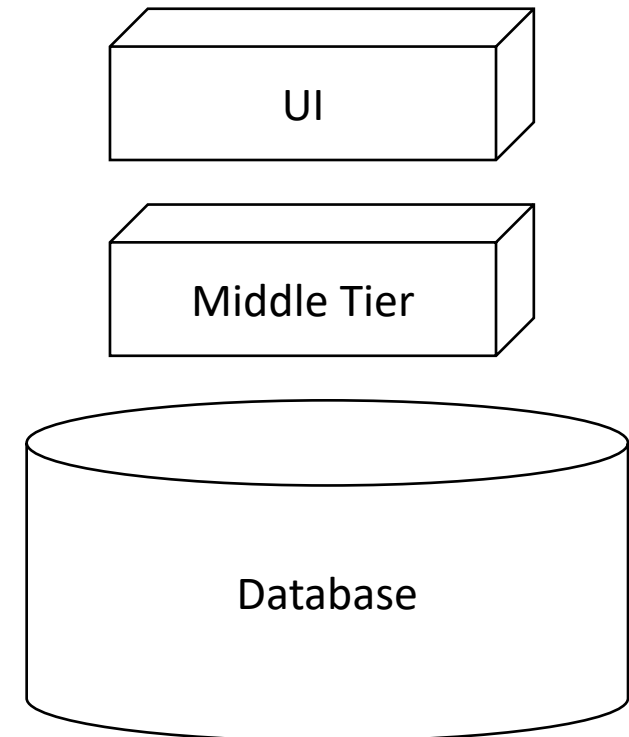
- **To meet all needs of data management**
  - Data Persistence (HA)
  - Data Access (Query Processing)
  - Correctness (Transaction Processing)
- **Two gold abstractions**
  - Declarative Query Language: **SQL / Relational Algebra**
  - Data Consistency: **Transaction**

*Are they still the **right abstractions** for modern App Dev ?*



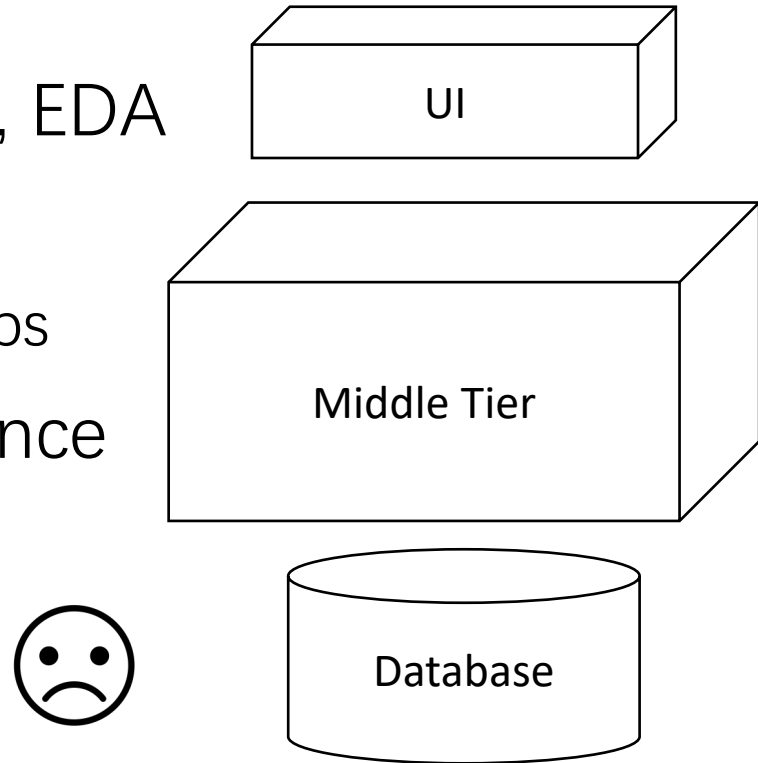
# The pre-Monolithic Age: pre 2000s

- Database Design: following ERD and NFs
- Data Access: SQL with Join
- Data Consistency: Transaction + XA(2PC)
- Business Logic in DB:
  - Heavy use of Stored Procedure
  - Heavy use of Trigger



# The Cloud Native Age: post 2010s

- DB Design ? : BLOB, Denormalization, NoSQL
- Data Access: ORM, No Join
- Data Consistency: Eventual Consistency, EDA
- Business Logic outside DB
  - Stored Procedure is forbidden by Web Apps
- Database is treated as a simple persistence layer!



# Uncle Bob: No DB



“As the lead developer, responsible for the quality of the software, my view of a relational database was that it would be a big, stogy, slow, expensive pain in the rear. We didn’t have complex queries. We didn’t need massive reporting capabilities. We certainly didn’t need a process with a multi-megabyte footprint sitting in memory and burning cycles. (Remember, this was the ‘80s). So I fought against this idea with everything I had; because it was the wrong technical solution.”

“The center of your application is not the database. Nor is it one or more of the frameworks you may be using. The center of your application are the use cases of your application.”

# Phenomena & Explanation

## Shift of Tradeoffs

# Object-Relational Impedance Mismatch

- Object-Relational Mapping (ORM)
  - Auto Mapping between Object & Relation
  - The de facto Standard of Database Access
    - 80%+ Projects on GitHub
  - But, ugly and raising new issues
    - 20% of cases have to circumvent ORM, causing significant pain in maintenance



# The No Join Phenomenon



- Most Tech teams forbid the use of Join.
- Why? Faster Iteration! Make maintenance easier!
  - Cannot afford dramatic performance degradation as data grows
  - Developers want to control rather than depending on DBAs
- Consequences
  - Data processing logic moves outside the DB
  - Prevalence of Denormalization and Redundancy
  - Poor Data Consistency
  - Wide Adoption of NoSQL, e.g., MongoDB

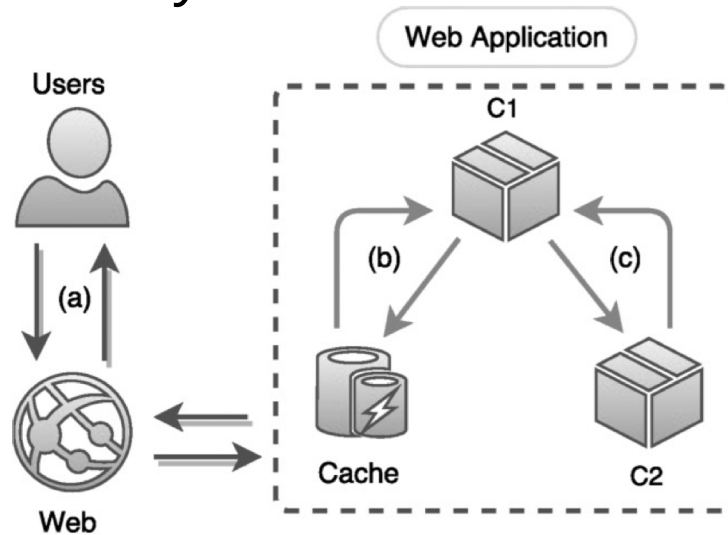
# Stored Procedure Abandoned



- Most Tech teams forbid the use of Stored Procedures
- Why? Faster Iteration! Make maintenance easier!
  - All business logics should stay in a single place: the middle tier
  - Facilitate Version Control, Testing and Debugging
- Consequences
  - Underutilization of Near Data Computation
  - Costly Data Movement
  - Poor Data Consistency

# Wide Application of App-Level Caches

- To boost performance, caching is regarded much more pragmatic than upgrading to a distributed DB.
  - Redis has become a default layer of a modern App
- Price to pay: Data Consistency!



(a) Process Steps.

```
public class C1() {
    public Object process() {
        //creating the cache component
        Cache cache = Cache.getInstance();

        //looking up for cache content (steps in b)
        Object content = cache.get("c1:c2-computation");
        if (content == null){
            //cache miss (steps in c)
            content = C2.compute();

            //caching the content for future requests
            cache.set("c1:c2-computation", content);
        }

        //doing some business logic...
        return content;
    }
}
```

(b) Code Example.

# A Survey of Cache Usage on Github

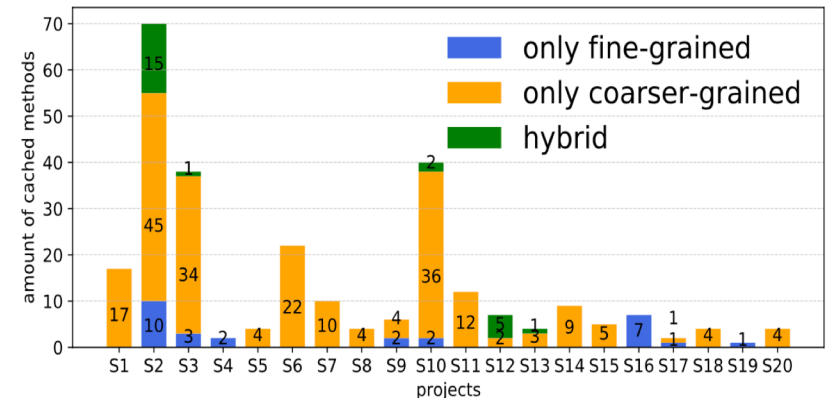
- 20 GitHub popular Spring Projects
- All use cache, but without consistency guarantees
- Why?

- Lack of link from data to cache entries!
- For instance, the following query is cached,

```
select * from fly_user  
where user_name=#{userName} limit 1;
```

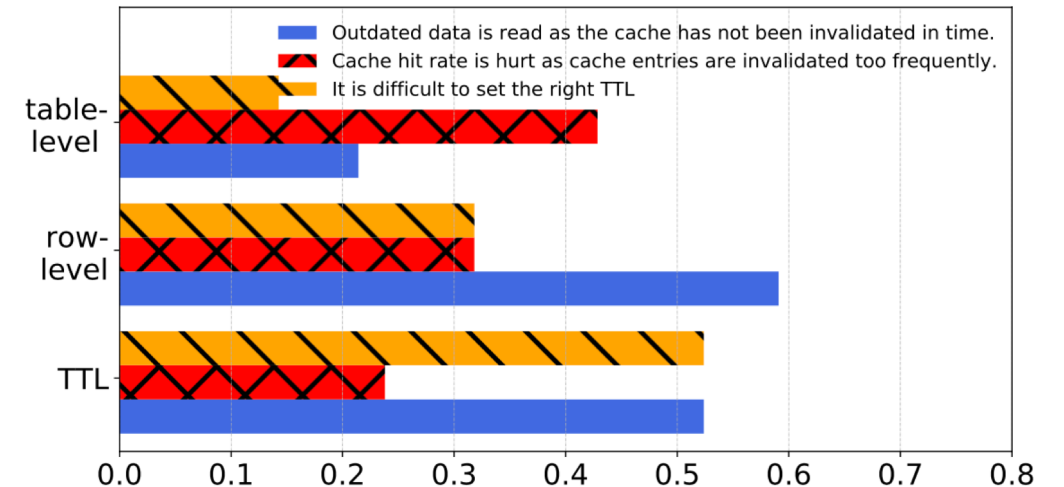
and we perform the following update.

```
update fly_user set avatar=#{avatar} where user_id=#{userId};
```



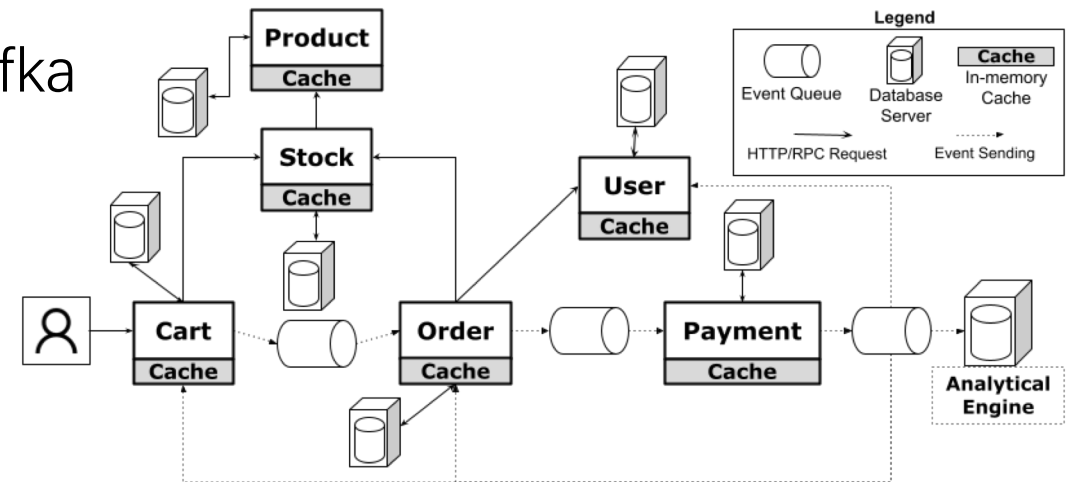
# Interview Results

- Interviewees: 64 experienced developers
- Feedbacks: App-level Cache is difficult to manage
  - Precise Cache Invalidation is hard
  - Conflict between Cache Efficiency and Data Consistency
  - Setting appropriate TTLs is an Art



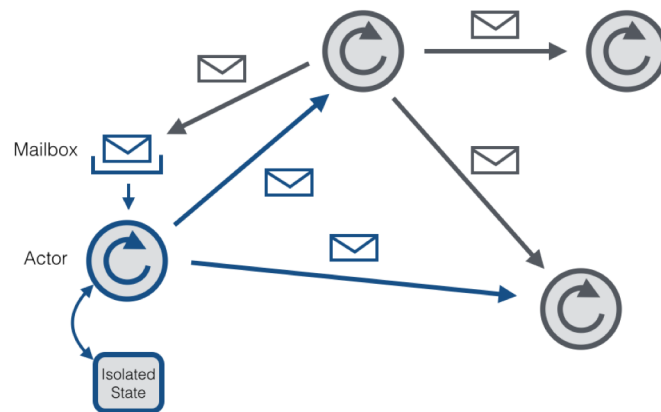
# Microservice & Event Driven Architecture

- An App is decomposed into Services, each with its own DevOps team
  - Separation among Services > Separation between App & DB
- Communication among Services via Asynchronized Events
  - Asynchronization → non-blocking !
  - Events all in message queues, e.g., Kafka
  - Cross Service Transactions
    - Eventual Consistency
    - Saga, not 2PC

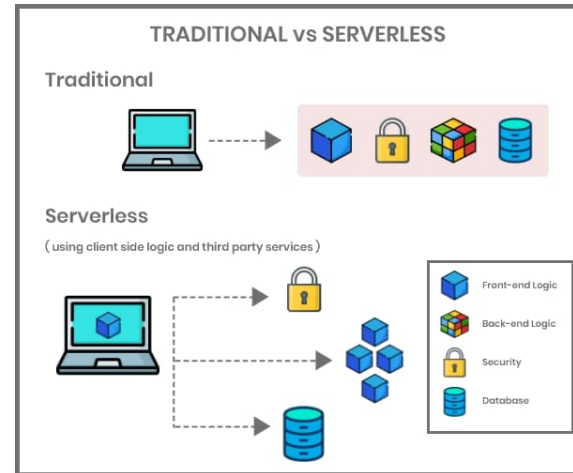


# Emerging Frameworks for taking it a step further

## The Actor Model

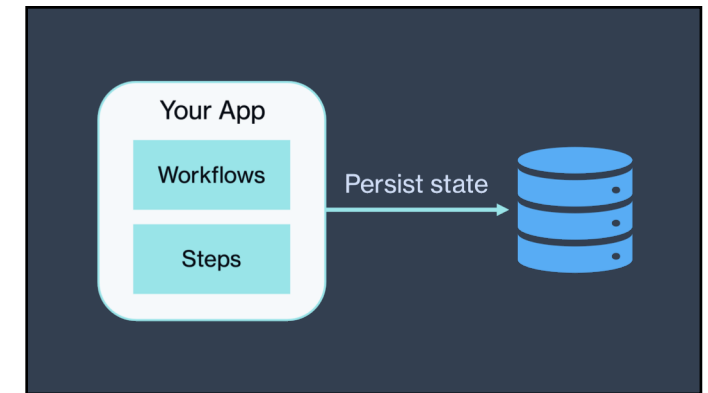


## FaaS



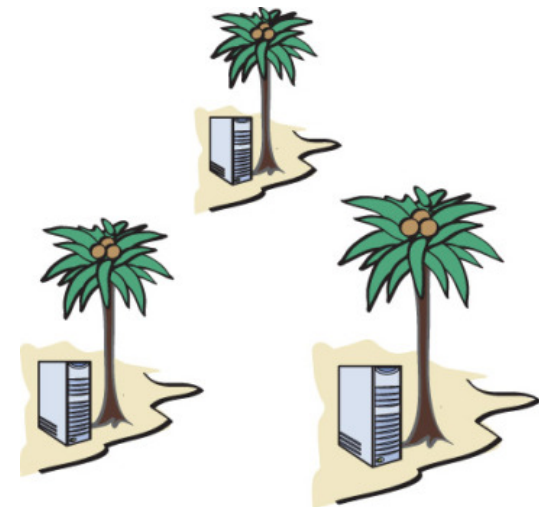
## DBOS

durable execution



# A Survey on Microservice Data Management

- Subjects: 9 popular projects on Github, 120 practitioners
- Results: 8 observations, 11 challenges,
  - Each Service maintains its own DB
  - Some data has to be duplicated across services
  - 1<sup>st</sup> pain point: Data Consistency
    - Solutions: Saga or ad hoc transactions
  - 2<sup>nd</sup> pain point: Cross Service Query
    - Solutions: ad hoc approaches





# Many Faces of Ad Hoc Transactions

By Chuzhe Tang, Zhaoguo Wang, Xiaodong Zhang, Qianmian Yu,  
Binyu Zang, Haibing Guan, and Haibo Chen

## ABSTRACT

Transactions are the fundamental database abstraction for ensuring application correctness under concurrency and failures. Using database transactions, developers are only tasked with demarcating critical business logic into transactions, and the underlying database system is responsible for coordinating their execution. Yet, in today's Web applications, transactions are often constructed in an ad hoc manner. That is, developers might explicitly use locking primitives or manual validation to coordinate critical code fragments, instead of relying on database transactions. We refer to such application-coordinated transactions as *ad hoc transactions*.

This paper presents the first comprehensive study on ad hoc transactions. Examining 91 ad hoc transaction instances taken from eight popular open source Web applications, we find that (i) every application studied uses ad hoc transactions (up to 16 per application), 71 of which play critical roles; (ii) compared with database transactions, coordination strategies of ad hoc transactions is much more flexible; (iii) ad hoc transactions are error-prone—53 of them have correctness issues, and 33 of them are confirmed by developers; and (iv) ad hoc transactions have the potential to improve performance in contentious

column uniqueness, through these APIs, and ORM frameworks check and report errors on invariant violations.

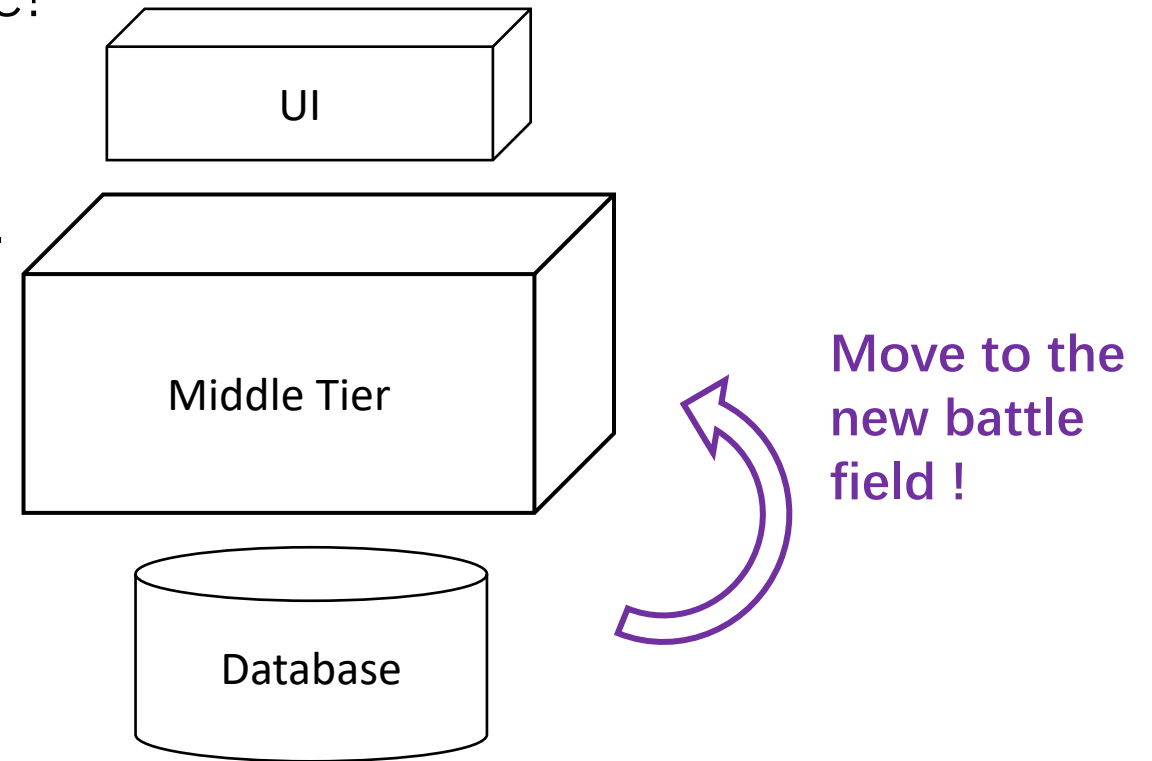
Besides these two approaches, we have observed that application developers often manually coordinate critical database operations. They might explicitly use locking primitives or validation procedures to implement concurrency control (CC) amid the application code. We refer to such ad hoc coordination of database operations as *ad hoc transactions*.

Figure 1 shows three real-world examples of ad hoc transactions from open source Web applications. In each example, the application code issues database operations via ORM frameworks and uses ad hoc constructs to coordinate them. The first two directly use locks for coordination, while the third one implements a validation-based protocol similar to optimistic concurrency control (OCC). We briefly explain Figure 1a, an add-to-cart code snippet from an e-commerce application. First, a lock identified by the cart ID is acquired and held until the whole business logic finishes. Then, database rows for the cart and corresponding items are retrieved and converted into runtime objects *cart* and *items* by the ORM. Next, a new item is added to the *items* collection, and the total price of the *cart* object is recalculated. Finally, the ORM persists two updated objects back to the database system.

Where to go?

# What does it mean to DBers?

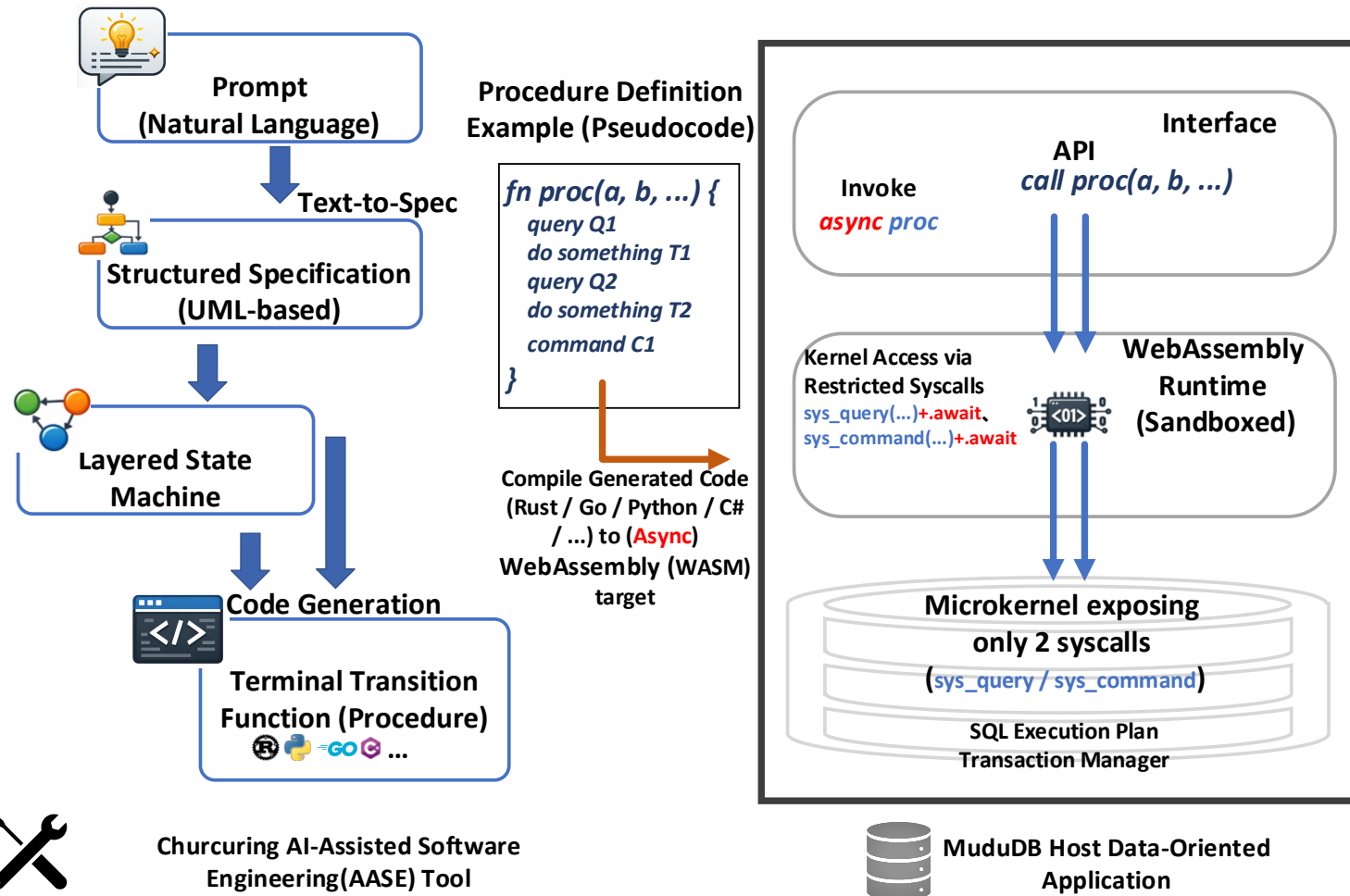
- The Abstractions provided by Relational DB are no longer adequate!
- Data moves outside the DB, which has become a pure persistence layer.
- But the middle tier has become increasingly bloated, presenting new opportunities
  - Data Consistency
  - Scalability
  - Efficiency



# Shift of tradeoff in the era of AI

- Tradeoff in pre-AI era: Iteration Speed > Cleanliness  
Human → Framework → Infrastructure
- Tradeoff in AI era: Iteration Speed < Cleanliness  
Human → Specification → AI → Code
- We will have
  - Cleaner App programs
  - Faster product delivery
  - Probably less abstraction layers

# An AI-Powered Spec-Driven Data-Centric Application Development & Operations Platform



<https://github.com/scuptio/churcuring>

<https://github.com/scuptio/mududb.git>

# AI-friendly Production Line

**Collapse unnecessary abstraction layers in AI-driven workflows**  
**Shorten the path from Idea → Prototype → Product**



**Concentrate entropy-reducing work at the leftmost stage of the production line.**

# Conclusion

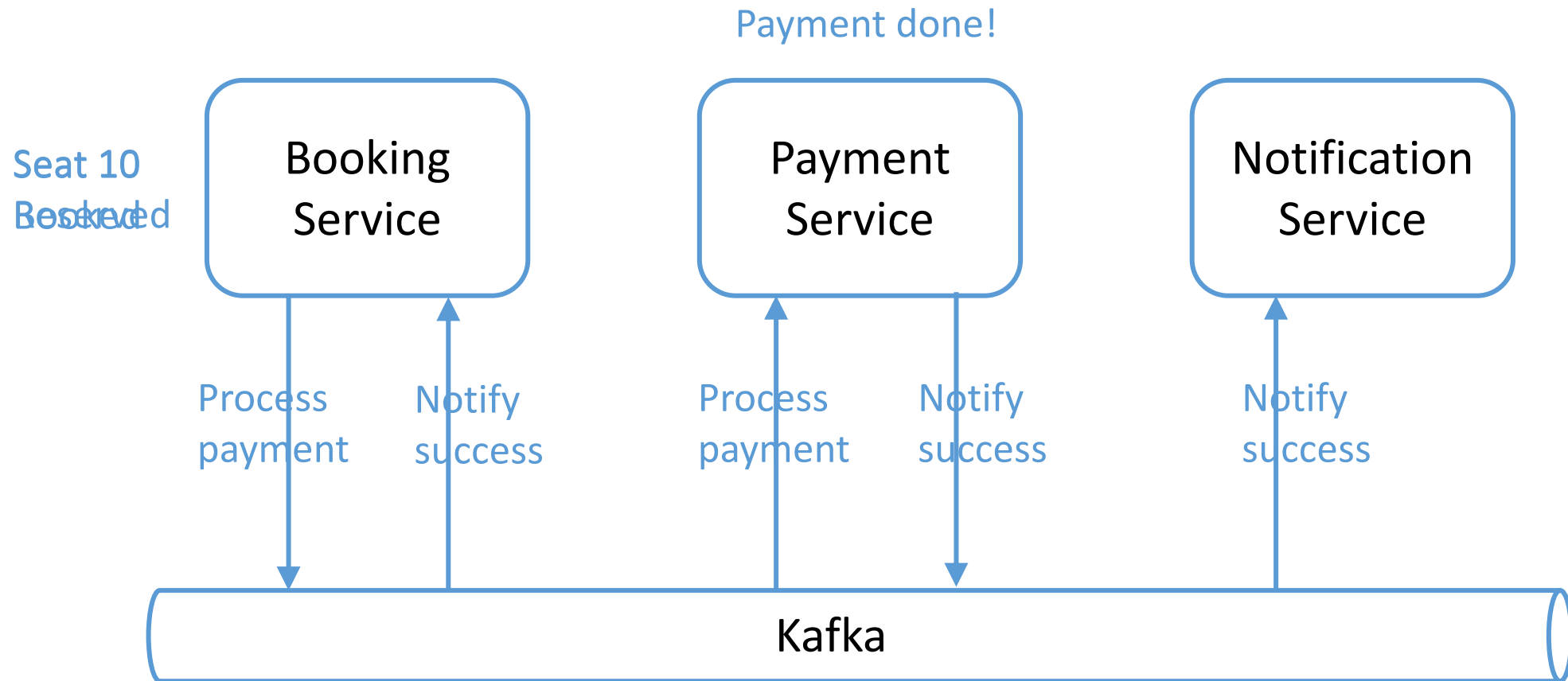
- The traditional prominence of the DBMS has diminished in the modern software stack.
- Data Processing Logic is moving outside the DBMS.
- The Middle Tier is presenting new opportunities.
- AI is shifting the tradeoffs again.



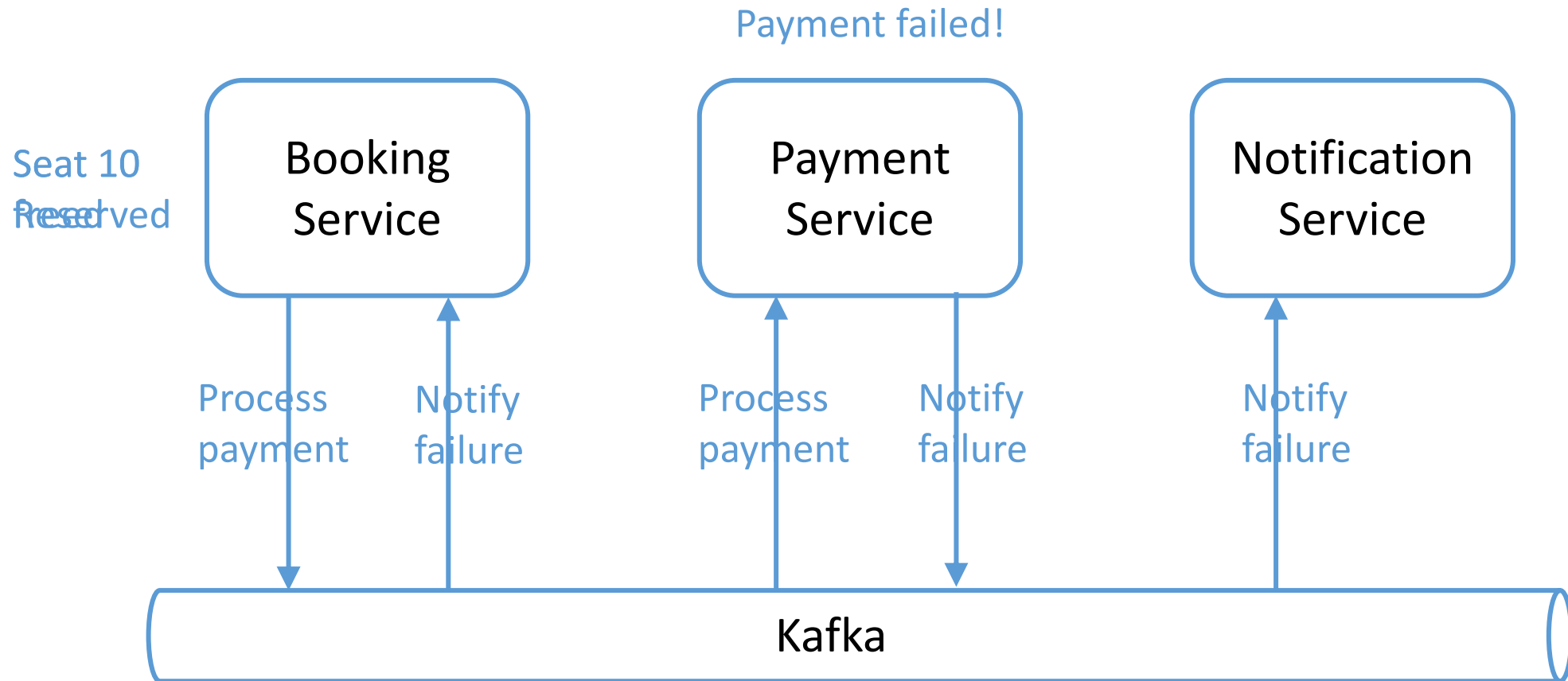
# Thanks!

xzhou@dase.ecnu.edu.cn

# Saga Transaction is not Easy!



# Saga Transaction is not Easy!



# Saga Transaction is not Easy!

Let's Retry

Seat 10  
Booked

